

Visual map of the factory-floor server, Cloudflare Worker, dispatch service, and how they all fit together. Auto-generated from the source tree at `v1.0.2`.

Node 18+ / Yarn 4 PnP

Express 5 + Socket.IO 4

Cloudflare Workers + D1 + R2

Local-first (offline-safe)

Bun runtime supported

PM2 / Windows Service

CONTENTS

1. What is this app?
2. High-level topology (3 lanes)
3. Runtime components
4. The factory server (server.js) — internal map
5. The Cloudflare Worker — internal map
6. Data flow: Excel → Catalog → Kiosk
7. Session lifecycle (sequence)
8. Dispatch service (material logistics)
9. Persistence & offline story
10. API surface
11. Security & deployment
12. Ops & maintenance
13. Risks & technical debt

1 What is this app?

AbaYa Track is a hybrid factory-floor productivity system for an abaya (Islamic garment) factory. Workers tap their employee code on a tablet, scan an abaya barcode, do their station's work (cutting, stitching, embroidery, buttons, stone work, invoice, packaging, QC), then tap Finish. The factory server times them inside configured shift windows and pushes events to a Cloudflare Worker so the owner (CEO) can read live dashboards and daily/weekly/monthly reports from any browser, anywhere.

Local-first. The factory *never* blocks on the cloud. If the internet drops, all floor operations keep working and event payloads are queued to a JSONL file that auto-replays when the link is back.

Three user groups. Factory operators (kiosk tablets), supervisors (LAN dashboard), executives (CEO cloud dashboard), plus IT/admin for installs and updates.

SERVER.JS

3,712 lines,
single file

EXPRESS ROUTES

~40 HTTP
endpoints

SOCKET EVENTS

5+5
client/server

WORKER ROUTES

~30
Cloudflare
endpoints

D1 TABLES

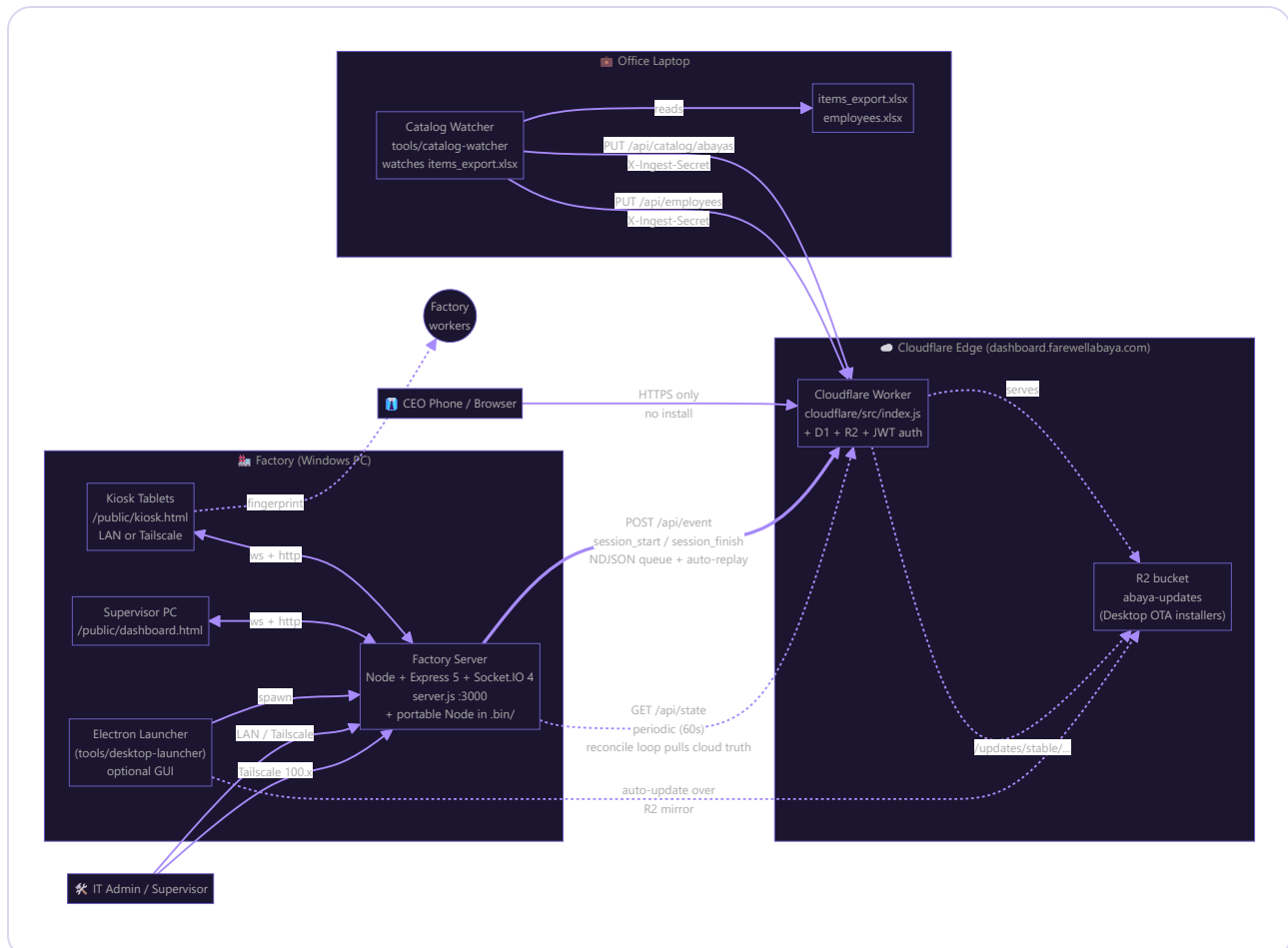
12+ via 10
migrations

WORK TYPES

12 factory
stations

2 High-level topology – three lanes

There are three independent ways to reach the system, each with a different trust boundary and a different runtime behind it.



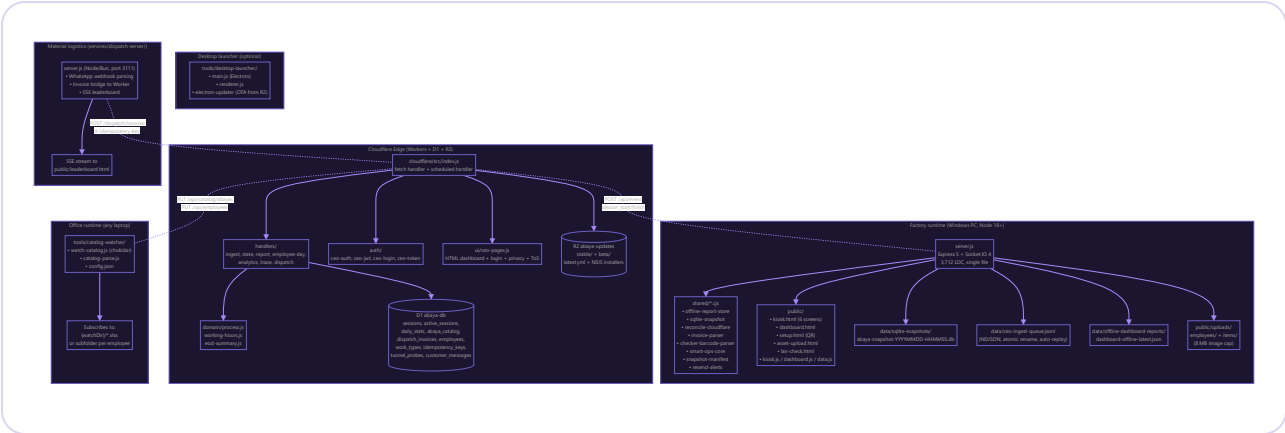
Three independent access lanes. Factory operations run entirely on-prem; the cloud is a read/analytics surface that is optional for daily work.

Lane summary

LANE	PATH	AUTH	INSTALL ON CLIENT
Kiosk (worker)	Browser → <code>http://LAN_IP:3000/kiosk.html</code> or <code>https://kiosk.farewellabaya.com</code> PWA	None (LAN-trusted / shared secret)	None (PWA add-to-home)
Supervisor / Admin	Browser → <code>http://LAN_IP:3000/dashboard.html</code> or Tailscale <code>100.x.x.x:3000</code>	None on LAN; Access policy on Tailscale serve	None (Tailscale client)
CEO	Browser → <code>https://dashboard.farewellabaya.com</code>	Password → JWT session cookie (CEO_JWT_SECRET)	Zero (browser only)
Office catalog sync	Watcher → <code>https://dashboard.farewellabaya.com/api/...</code>	X-Ingest-Secret shared key	Yarn + watcher config

3 Runtime components

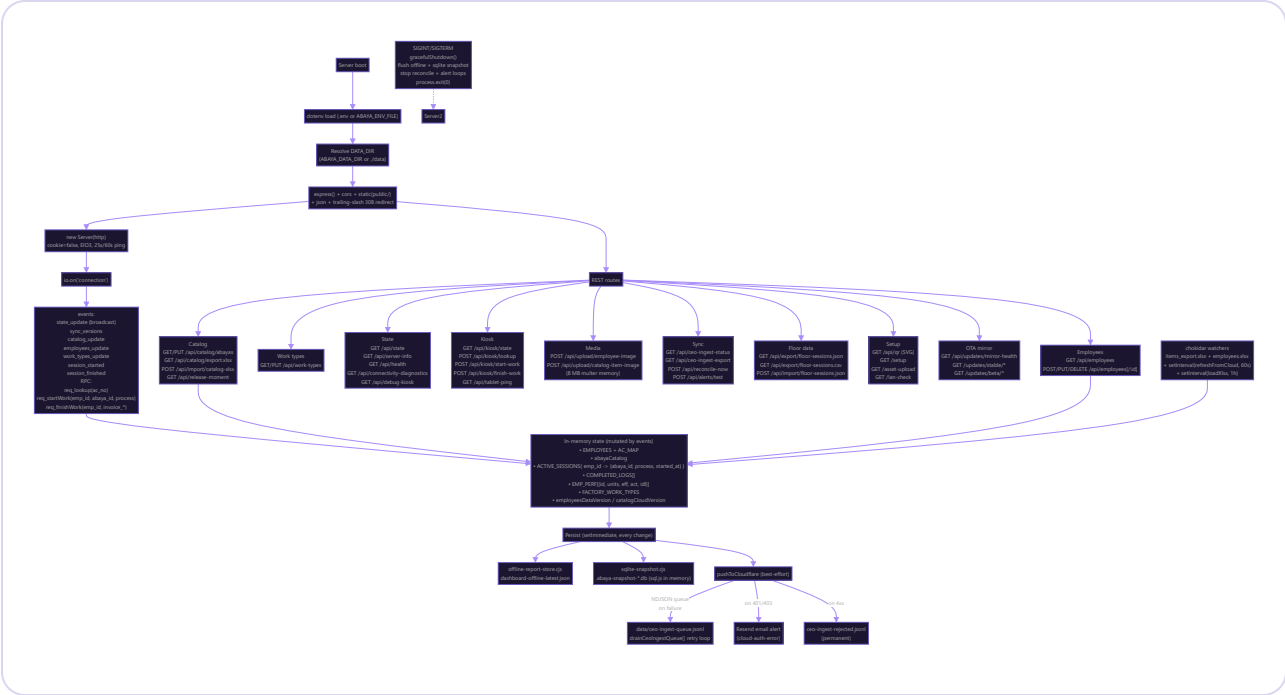
Five independent deployables. Each one is optional except the first one.



Five deployables. Solid arrows = in-process dependencies. Dotted arrows = HTTPS across network boundaries.

4 The factory server (server.js) – internal map

The factory server is one 3,712-line Node file. Internally it acts like a mini-framework: env loading, data store, Excel parsers, Watcher, Cloudflare push, Socket.IO, REST routes, media upload, shutdown hooks.



server.js as a single-process mini framework. The 5 colored clusters are: boot, in-memory store, routes, Socket.IO, persistence. The dotted arrow on graceful shutdown fires on Ctrl+C / LAUNCH-ALL.

HTTP routes — quick reference

METHOD	PATH	PURPOSE
GET	/api/state	Bounded realtime snapshot (active sessions + recent logs + perf)
GET	/api/employees	Employee directory (JSON, versioned)
PUT/POST/DELETE	/api/employees[:id]	CRUD on roster (with integrity check)
GET/PUT	/api/catalog/abayas	Catalog cache (auth via X-Ingest-Secret)
POST	/api/import/catalog-xlsx	Drag-and-drop Excel import (multer, 20 MB cap)
GET	/api/catalog/export.xlsx	Download current catalog as Excel
GET/PUT	/api/work-types	Factory work types (Tailor (01), Cutting, Stone Work, ...)
GET	/api/server-info	LAN IP + port + uptime + boot id
GET	/api/connectivity-diagnostics	Self-checks for tablet LAN troubleshooting

METHOD	PATH	PURPOSE
GET	/api/health	Liveness
POST	/api/kiosk/lookup	Identify by AC number (REST mirror of req_lookup)
POST	/api/kiosk/start-work	REST mirror of req_startWork
POST	/api/kiosk/finish-work	REST mirror of req_finishWork
POST	/api/upload/employee-image	multer memory upload → public/uploads/employees/
POST	/api/upload/catalog-item-image	Same for items
GET	/api/ceo-ingest-status	Pending queue depth, last success, alert state
GET	/api/ceo-ingest-export	Manual NDJSON download (X-Ingest-Secret)
POST	/api/reconcile-now	Force a reconcile cycle
POST	/api/alerts/test	Send a test Resend email
GET	/api/export/floor-sessions.json csv	Floor session export (X-Export-Secret)
POST	/api/import/floor-sessions.json	Restore from floor session backup
GET	/api/qr?url=&size=	QR SVG for tablet onboarding
GET	/updates/stable/<file> and /updates/beta/<file>	LAN mirror of Electron installers (latest.yml + NSIS)

5 The Cloudflare Worker — internal map

Deployed to dashboard.farewellabaya.com via wrangler.jsonc at the repo root. Single fetch handler with conditional routing; scheduled handler runs crons.



Syntax error in text

mermaid version 10.9.1

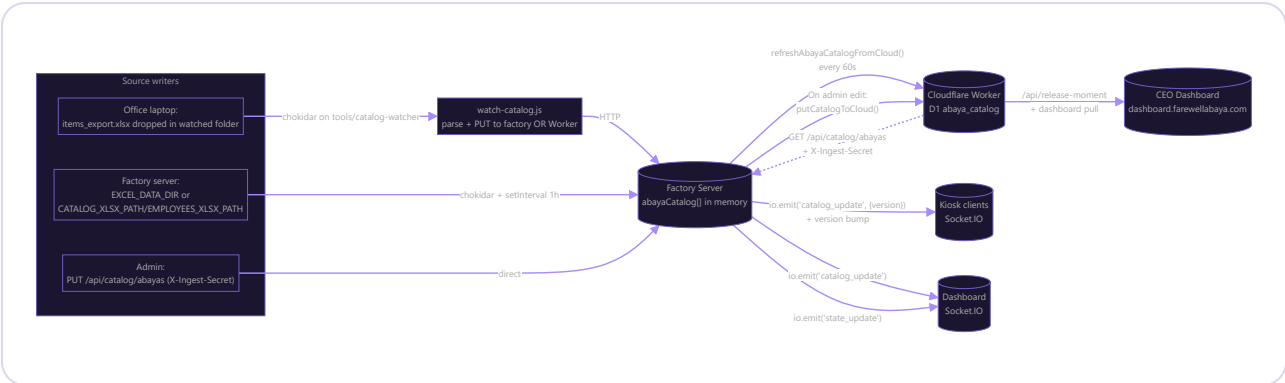
Worker is a single fetch handler with conditional routing. CEO routes are gated by JWT cookie + rate-limit; ingest/catalog/roster use the X-Ingest-Secret shared key so the factory server can push without a CEO login.

Worker secrets & bindings (wrangler.jsonc)

TYPE	NAME	PURPOSE
D1	DB (abaya-db, 5d61fb15-...)	All relational state
R2	UPDATES (abaya-updates)	OTA installers, latest.yml
Secret	CEO_PASSWORD	Login password (env: CEO_PASSWORD)
Secret	CEO_JWT_SECRET	HMAC sign session + refresh tokens
Secret	INGEST_SECRET	X-Ingest-Secret from factory server
Secret	WHATSAPP_TOKEN , WHATSAPP_VERIFY_TOKEN , WHATSAPP_PHONE_NUMBER_ID	Meta Cloud API for dispatch.js
Secret	DISPATCH_BRIDGE_SECRET	Local dispatch server ↔ Worker
Rate limit	INGEST_RATE_LIMIT 3000/60s	Factory ingest throttle
Rate limit	CATALOG_PUT_RATE_LIMIT 120/60s	Catalog writes
Rate limit	CEO_READ_RATE_LIMIT 800/60s	CEO dashboard polling
Cron	0 14 * * *	EOD summary email
Cron	* * * * *	Tunnel health probe
Route	dashboard.farewellabaya.com (custom domain)	Public surface

6 Data flow — from Excel to Kiosk

There are three writer paths for the catalog. The factory server merges them and broadcasts the latest version to all kiosk/dashboard clients.



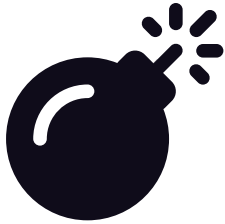
Three writer paths converge on the in-memory catalog; the server polls the cloud every 60s and pushes back on admin edits. Last-writer-wins by version timestamp.

Factory data — local-first sources of truth

WHAT	SOURCE (PRECEDENCE)	REFRESH
Abaya catalog	1. <code>CATALOG_XLSX_PATH</code> (Excel) → 2. Cloud D1 (via ingest) → 3. <code>DEFAULT_ABAYA_CATALOG</code> (built-in demo)	chokidar + setInterval (1h) + cloud pull (60s)
Employees	1. <code>EMPLOYEES_XLSX_PATH</code> → 2. <code>data/employees-manual.json</code> (manual edit) → 3. Cloud D1 (fresh-install seed only) → 4. <code>DEFAULT_EMPLOYEES</code>	chokidar + setInterval (1h) + cloud pull (60s)
Work types	1. <code>data/work-types.json</code> → 2. Cloud D1 (fresh-install seed) → 3. <code>DEFAULT_FACTORY_WORK_TYPES</code>	Cloud pull (60s)
Working hours	1. Cloud D1 <code>worker_settings</code> → 2. <code>defaultWorkingHoursConfigLocal()</code>	Cloud pull (5 min)

7 Session lifecycle (sequence)

The core unit of work. A worker taps in, picks an abaya, does their station, taps out. Duration is computed only against configured shift windows so breaks and off-hours never inflate the numbers.

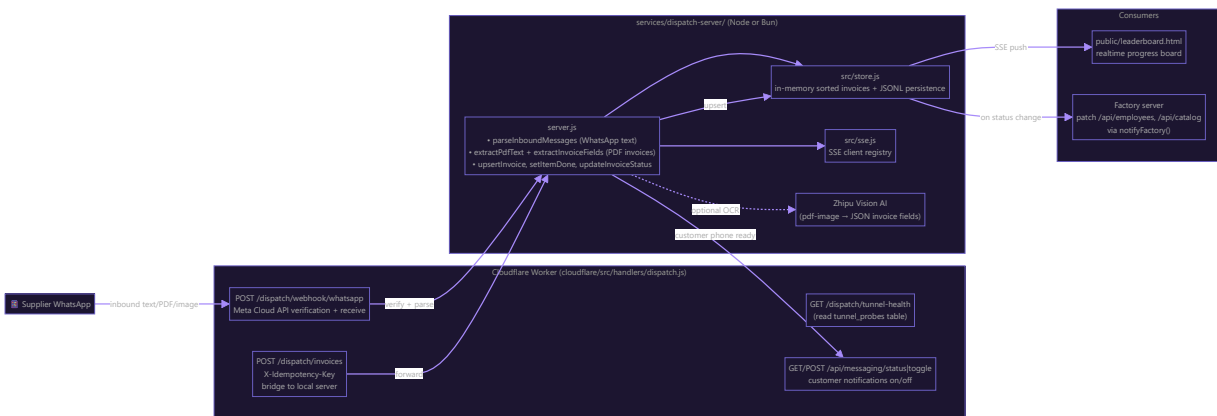


Syntax error in text mermaid version 10.9.1

One Start ties a worker to one abaya and one process until they tap Finish. Breaks inside the shift window are counted; off-hours are not.

8 Dispatch service (material logistics)

A separate *optional* sub-service in `services/dispatch-server/` for material/invoice logistics. It is a Bun/Node HTTP server (default port 3111) that runs alongside the main factory server.

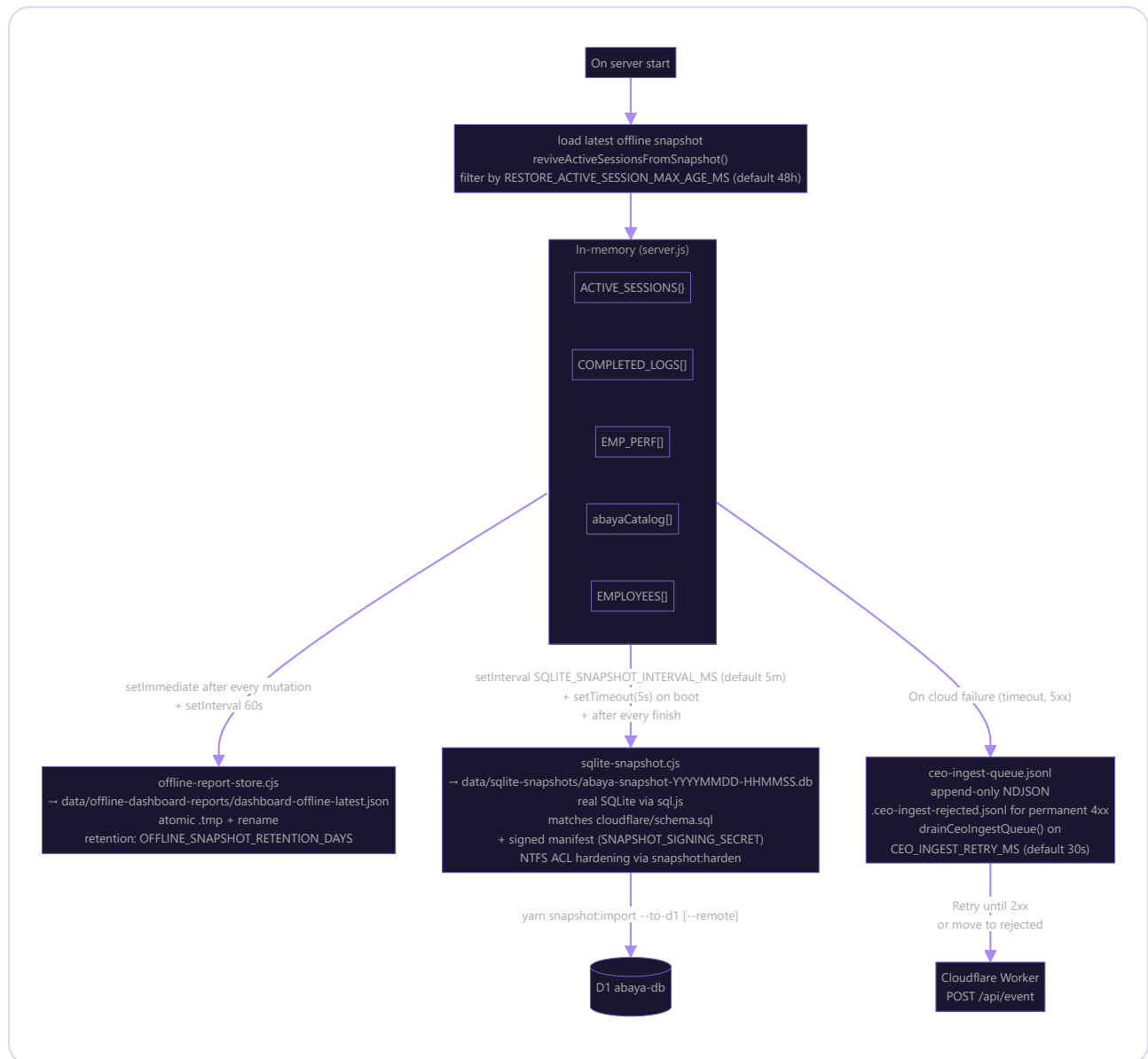


Hybrid-edge material flow. Incoming WhatsApp/PDF is processed at the cloud edge first (faster webhook ack), then mirrored to the local server for real-time SSE to the floor leaderboard.

Note on scope. The dispatch service is intentionally isolated from the main factory server. They share `schema.sql` and the `dispatch_invoices` migration, but the main Worker handles the cloud-side webhook and bridge; `services/dispatch-server/` only runs on the factory PC. The README explicitly warns: "Do not run `wrangler deploy` from this folder."

9 Persistence & offline story

Three independent local stores back up the in-memory state, plus a fourth offline-resilient queue for cloud writes.



Layered durability: In-memory is fast but volatile; offline JSON is human-readable; SQLite is the canonical local copy; the NDJSON queue survives full power loss and reorders gracefully.

Restore rules on startup

- **In-progress sessions** (kiosk) are revived from the latest snapshot if `started_at` is within `RESTORE_ACTIVE_SESSION_MAX_AGE_MS` (default 48h) *and* the employee still exists in the current roster.
- **Completed logs** are restored subject to `OFFLINE_LOG_WINDOW_MS` (default = entire history, still capped by `STATE_LOG_MAX_ROWS`).
- **Catalog & roster** are always re-fetched from the cloud (60s) and reloaded from the local xlsx (1h) on every boot — never trusted from the offline snapshot.
- **Graceful shutdown** (`SIGINT/SIGTERM`) flushes both `persistOfflineDashboardReport()` and `persistSqliteSnapshot()` before `process.exit(0)`. An OS power-cut may still lose the last in-progress session.

10

API surface — sockets and events

Socket.IO (factory server)

DIRECTION	EVENT	PAYLOAD
S→C	state_update	Bounded realtime snapshot
S→C	sync_versions	employees / catalog / workTypes / workingHours versions
S→C	catalog_update	{ version: string }
S→C	employees_update	{ count, version }
S→C	work_types_update	{ workTypes, version }
S→C	session_started	{ employee, session }
S→C	session_finished	{ employee, session }
C→S	req_lookup(ac_no, cb)	Returns {ok, employee, is_active, abaya_code, session_process, session_started_at}
C→S	req_startWork({emp_id, abaya_id, process}, cb)	Returns {ok, log_id}
C→S	req_finishWork(payload, cb)	Returns {ok, log_id, duration_sec}

Socket.IO (kiosk — fallback only)

The same Socket.IO endpoint is also used by `public/kiosk.js` directly. The dashboard prefers Socket.IO but falls back to `GET /api/state` polling on disconnect.

Cloudflare Worker

METHOD	PATH	AUTH
POST	/api/ceo/session	password in body
POST	/api/ceo/session/refresh	refresh cookie
POST	/api/ceo/logout	none
GET/PUT	/api/catalog/abayas	open GET, X-Ingest-Secret PUT

METHOD	PATH	AUTH
GET/PUT	/api/employees	open GET, X-Ingest-Secret PUT
GET/PUT	/api/work-types	open GET, X-Ingest-Secret PUT
GET/PUT	/api/settings/working-hours	CEO OR ingest-secret
POST	/api/event	X-Ingest-Secret
GET	/api/state	CEO cookie
GET	/api/report?type=Daily Weekly Monthly	CEO
GET	/api/report/employee-day	CEO
GET	/api/analytics	CEO
GET	/api/trace?barcode=...	CEO
GET	/api/release-moment	CEO
GET	/api/messaging/status	CEO
POST	/api/messaging/toggle	CEO
GET/POST/PATCH	/dispatch/invoices	DISPATCH_BRIDGE_SECRET
GET/POST	/dispatch/webhook/whatsapp	Meta verify token
GET	/dispatch/tunnel-health	CEO

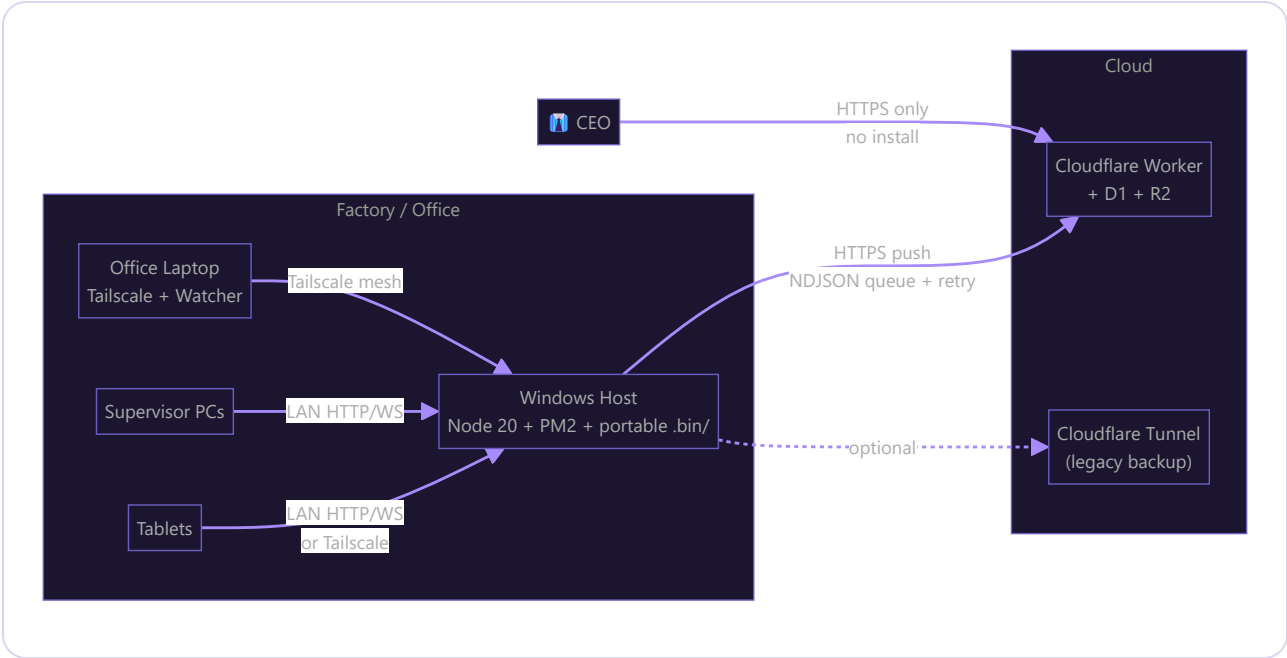
11 Security & deployment

Auth model

SURFACE	AUTH
Factory kiosk / dashboard (LAN)	None by default (LAN-trusted); optional ASSET_UPLOAD_SECRET on media uploads
Factory ingest (server→Worker)	X-Ingest-Secret shared with Worker env

SURFACE	AUTH
Factory catalog/roster PUT (admin)	<code>X-Ingest-Secret</code> or <code>CATALOG_UPLOAD_PASS</code>
Floor data export (CSV/JSON)	<code>X-Export-Secret</code> = <code>FLOOR_EXPORT_SECRET</code> or fallback ingest secret
CEO dashboard (Worker)	Password login → JWT session (HttpOnly+Secure cookie) + refresh token rotation; <code>CEO_JWT_SECRET</code> signs
Worker→Worker bridge (dispatch)	<code>DISPATCH_BRIDGE_SECRET</code>
Network boundary	Tailscale mesh (WireGuard) preferred; Cloudflare Tunnel as legacy backup

Deployment topology



12 Ops & maintenance

Day-to-day operating commands. The repo's `START_HERE.txt` and `START-HERE-BUILD-INSTRUCTIONS.txt` walk through these.

Install / launch

- **First install:** `install\INSTALL.bat` (or `yarn setup`) — Corepack, Yarn 4 PnP deps, .env bootstrap, Desktop shortcut. No PowerShell required for the install itself.
- **Daily start (classic):** `install\LAUNCH-ALL.bat` — optional Cloudflare tunnel, runtime picker (Node or Bun), server + watcher in separate consoles, auto-start at Windows login.
- **Always-on (PM2):** `powershell -NoProfile -ExecutionPolicy Bypass -File install\SETUP-PM2-BOOT.ps1` once; then `pm2 status`, `pm2 logs abaya-server`, `pm2 reload all`.
- **Electron GUI (optional):** `tools/desktop-launcher` with `electron-updater` OTA from R2. Same processes, one window.
- **Cloud deploy:** `yarn deploy:all` or `install\DEPLOY-ALL.ps1` (Worker + Pages + tunnel).
- **Single-EXE installer:** `yarn package:installer:win` produces an NSIS installer under `dist/desktop-launcher/`.

Snapshots & integrity

- **Manual snapshot:** `yarn snapshot:db` writes a fresh `abaya-snapshot-*.db`.
- **Info:** `yarn snapshot:info` shows metadata.
- **Import to D1:** `yarn snapshot:import --to-d1 [--remote]`.
- **Tamper protection:** set `SNAPSHOT_SIGNING_SECRET`, then `yarn snapshot:hardening` (NTFS ACLs) and `yarn snapshot:verify` in a scheduled task.
- **Tests:** `yarn test` (unit), `yarn test:system` (smoke), `yarn test:offline-store`, `yarn qa:qc`, `yarn verify:employee-xlsx`.
- **Live eval:** `yarn evaluate:rollout` / `yarn evaluate:live`.
- **CEO state check:** `yarn verify:ceo`.

13 Risks & technical debt (worth knowing)

From `docs/SYSTEM_DESIGN.md` §12 and a fresh scan of the source.

- **Monolith** `server.js` (3,712 lines). Easy to read, hard to unit-test. Routes, persistence, push, watchers, alerts all share one closure.
- **No horizontal scale** — single Node process, in-memory state. Adding a second node would require moving state out (Redis or always-on DB).
- **OS power-cut window.** The default 60s `persistOfflineDashboardReport` cadence can lose the last minute of work. The 48h restore window is generous but not infinite.

- **No build pipeline for the client.** `public/kiosk.js` and `public/dashboard.js` are loaded directly. No minification, no module bundling, no source maps.
- **Factory socket/read APIs are unauthenticated by default.** Security relies on the LAN boundary (or Tailscale). `CORS: *` is enabled.
- **CEO token can land in query string** (bootstrap via `?token=`). Reduces to cookie after the first 302, but the URL ends up in browser history and referer logs.
- **No secret rotation or audit trail.** Static `CF_INGEST_SECRET`, `CEO_JWT_SECRET`, `ASSET_UPLOAD_SECRET`.
- **Multi-writer catalog** (Watcher vs Excel vs Admin PUT) can race. Last-writer-wins by version timestamp; no CRDT.
- **Domain constants duplicated** across `public/data.js`, `cloudflare/src/domain/process.js`, `server.js` 's `DEFAULT_FACTORY_WORK_TYPES`, and `tools/catalog-watcher/config.json`. Drift risk.
- **`.env.example` has unresolved git merge markers** at lines 123–139 (`<<<<<< HEAD / ===== / >>>>>> 10247c8...`) around the `ABAYA_UPDATE_MIRROR_BASE_URL` docs. Cosmetic but worth a clean resolve next time you touch that file.
- **Stale `1` and `rows` files** at the repo root (both 0 bytes) — leftover from a paste / Excel export. Not a bug, but they will show up in `git status`.
- **`items_export.xlsx` and other samples** sit in the repo root; they're convenient for manual testing but probably shouldn't be tracked long-term.

Where the system shines

- **Offline resilience.** NDJSON queue + json snapshot + sqlite snapshot = three independent durability layers, plus a reconcile loop that pulls cloud truth into the factory.
- **Local-first by design.** Cloud outage never affects floor operations.
- **Portable single-folder install.** `install\LAUNCH-ALL.bat` downloads a portable Node 20, sets up deps, creates a Windows Startup shortcut — runs from a USB stick if you need it to.
- **Defense in depth on persistence.** Tamper-protected SQLite snapshots, NTFS ACL hardening, signed manifest, scheduled verification.
- **Three independent deployment shapes:** batch launcher (classic), PM2 (always-on), Electron (GUI with OTA updates).

Generated from the v1.0.2 source tree. Diagrams rendered with Mermaid 10. Source: `docs/system-diagrams/index.html`.